Assignment 2: Agent Design Document

Personal Productivity AI Agent

AI Engineering Fellowship 2026 – Week 3

Student: Hassan Murtaza Zaidi

---

1. Problem Statement

Modern professionals and students manage numerous daily tasks, notes, meetings, and deadlines. Traditional task management applications require users to manually organize information and cannot intelligently decide which action to perform based on natural language instructions.

The Personal Productivity AI Agent solves this problem by allowing users to communicate using natural language. The agent understands user intent, decides whether an AI response or a software tool is required, requests human approval for sensitive operations, executes tools, stores information, and returns structured results.

The system combines Large Language Models, LangGraph workflows, semantic search, tool calling, and persistent storage to create an intelligent productivity assistant.

2. Users

The system is intended for:

- University students
- Software developers
- Project managers
- Researchers
- Freelancers
- Office employees
- Anyone managing personal tasks and notes

3. Major Use Cases

The agent supports the following requests:

Task Management

- Create tasks
- List tasks
- Update existing tasks
- Complete tasks
- Delete tasks

Notes Management

- Save notes
- Search notes using semantic similarity

Productivity

- Generate daily work plans
- Extract meeting action items
- Answer general productivity questions

4. Agent Responsibilities

The agent is responsible for:

- Understanding natural language requests
- Selecting the correct tool
- Extracting tool arguments
- Managing conversation state
- Executing productivity tools
- Requesting human approval before destructive operations
- Handling tool failures gracefully
- Logging execution activities
- Maintaining session memory
- Returning structured responses

5. Agent Boundaries

The agent is NOT allowed to:

- Execute destructive actions without user approval
- Access private files outside the project
- Modify operating system settings
- Execute arbitrary Python code
- Access unauthorized APIs
- Reveal environment variables or API keys
- Perform actions outside the registered tool set

6. Tool Catalogue

The project currently includes the following tools.

Tool	Purpose
create_task	Create new productivity task
list_tasks	Display tasks
update_task	Modify task details
complete_task	Mark task as completed
delete_task	Delete an existing task
save_note	Store notes
search_notes	Semantic note search
extract_meeting_actions	Extract meeting summary and action items
generate_work_plan	Generate daily productivity schedule

7. State Model

The system uses LangGraph to maintain execution state.

The state contains:

- Conversation messages
- Selected tool
- Tool arguments
- Tool execution result
- Pending approval action
- Approval status
- Execution errors
- Next workflow node

The AgentController also maintains:

- Session memory
- Pending approval request
- Activity log
- Execution limits
- Conversation history

State persistence allows the conversation to continue across multiple requests during the same Streamlit session.

8. Approval Model

Certain operations modify stored data and therefore require explicit human approval.

Approval-required tools include:

- update_task
- complete_task
- delete_task

Approval workflow:

1. User requests an operation.
2. LangGraph identifies a sensitive tool.
3. The workflow pauses.

4. The agent asks:

"Do you want me to continue?"

5. User replies:
    - yes
    - no
6. Only after approval does the tool execute.

This design prevents accidental destructive actions.

9. Error Handling Strategy

The agent follows multiple layers of error handling.

Input Validation

Tool arguments are validated before execution.

Tool Errors

Exceptions are caught inside the Tool Executor.

Database Errors

Missing tasks or invalid IDs return friendly messages.

LangGraph Errors

Decision node failures return a safe fallback response.

Response Formatting

Errors are converted into readable user messages rather than raw Python exceptions.

Example:

❌ Action failed

Task not found

10. Security Considerations

Several security mechanisms are implemented.

API Key Protection

Secrets are stored inside environment variables using a .env file.

API keys are never hard-coded.

Human Approval

Sensitive write operations require user approval before execution.

Tool Isolation

Only registered tools may be executed.

Unknown tool names are rejected.

Session Memory

Conversation state is isolated to the active Streamlit session.

Data Storage

Tasks are stored in JSON format.

Semantic note embeddings are stored inside ChromaDB.

Prompt Safety

The LLM is instructed to return structured JSON decisions.

Only validated tool names are accepted.

Error Sanitization

Internal exceptions are hidden from users.

Only safe, readable error messages are displayed.

11. Overall System Workflow

The execution flow is:

1. User enters a natural language request.
2. LangGraph Decision Node determines the correct action.
3. If no tool is needed, the LLM responds directly.
4. If a tool is required:
    - Tool arguments are generated.
    - Approval is requested if necessary.
    - The tool executes.
5. Results are formatted.
6. Activity is logged.
7. Session state is updated.
8. Response is returned to the user.

12. Technologies Used

- Python
- LangGraph
- LangChain
- OpenRouter GPT-4o-mini
- ChromaDB
- Sentence Transformers
- Streamlit
- JSON Database
- dotenv

13. Conclusion

The Personal Productivity AI Agent demonstrates a complete tool-using AI workflow built with modern AI engineering practices. It combines Large Language Models with structured decision

making, persistent state management, semantic search, approval workflows, execution logging, and reusable productivity tools.

The modular architecture allows future extensions such as calendar integration, email automation, cloud databases, and multi-agent collaboration while maintaining security, reliability, and maintainability.